

SISTEMAS OPERACIONAIS

Deadlocks

Material de Estudo com Simulado

O que é um Deadlock?

Um deadlock (impasse) ocorre quando um conjunto de processos fica bloqueado permanentemente porque cada processo espera por um recurso que está retido por outro processo do mesmo conjunto. Nenhum processo consegue progredir — o sistema entra em estado de espera circular infinita.

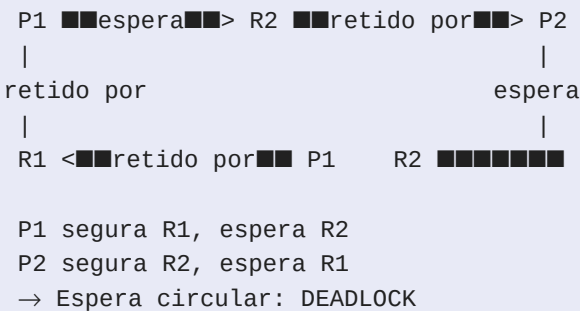


Figura 1 — Grafo de alocação de recursos com ciclo: P1P2. Ciclo = deadlock potencial.

Dica: Deadlock é diferente de starvation: em starvation um processo nunca é escalonado (por baixa prioridade), mas poderia progredir se fosse. Em deadlock, nenhum dos processos envolvidos pode progredir de forma alguma.

As 4 Condições de Coffman

Coffman et al. (1971) provaram que deadlock só pode ocorrer se TODAS as quatro condições abaixo existirem simultaneamente. Negar qualquer uma delas é suficiente para prevenir deadlock.

#	Condição	Descrição	Exemplo
1	Exclusão Mútua	Recurso não pode ser compartilhado, apenas 1 processo por vez	Impedida, mutex
2	Hold and Wait	Processo retém recursos e solicita outros	P1 segura mutex_A, espera mutex_B
3	No Preemption	Recursos não podem ser retirados de um processo	Se não pode forçar P1 a liberar mutex_A
4	Espera Circular	Ciclo de espera: P1→P2→P1	P1 espera P2, P2 espera P1

Dica: Para a prova: Coffman = Exclusão Mútua + Hold and Wait + No Preemption + Espera Circular. As 4 condições são NECESSÁRIAS mas não suficientes individualmente — todas precisam ocorrer juntas.

Estratégias de Tratamento

1. Prevenção (Prevention)

Garante que pelo menos uma das 4 condições nunca ocorra. Abordagem estática — impõe restrições ao sistema antes de executar:

- Negar Hold and Wait: processo deve requisitar TODOS os recursos antes de iniciar (impraticável, desperdício)
- Negar No Preemption: se processo não consegue recurso, libera tudo que tem e tenta novamente
- Negar Espera Circular: ordenar recursos globalmente ($R1 < R2 < R3$) e requisitar sempre em ordem crescente

2. Esquiva / Evitação (Avoidance)

O SO avalia dinamicamente cada requisição de recurso e decide se pode atendê-la sem entrar em estado inseguro. Requer conhecimento prévio da demanda máxima de cada processo.

Algoritmo do Banqueiro (Banker's Algorithm — Dijkstra)

Metáfora: um banco (SO) empresta recursos (dinheiro) a clientes (processos) sabendo que cada um devolverá tudo eventualmente. O banco mantém estado SEGURO se existe pelo menos uma sequência de atendimento em que todos os processos terminam. Se uma requisição levaria ao estado inseguro, ela é bloqueada até que seja seguro atendê-la.

```
/* Estruturas do Banker's Algorithm */
int disponivel[M];      // recursos disponíveis
int maximo[N][M];      // demanda máxima de cada processo
int alocado[N][M];     // alocação atual
int necessidade[N][M]; // necessidade[i][j] = maximo - alocado

/* Estado seguro = existe sequencia S tal que
   todos os processos em S podem terminar
   com os recursos disponíveis + liberados */
```

Código 1 — Estruturas de dados do Banker's Algorithm. N=processos, M=tipos de recurso.

3. Detecção e Recuperação (Detection & Recovery)

Permite que deadlock ocorra, mas detecta periodicamente usando o grafo de alocação de recursos (ou algoritmo equivalente ao Banker sem estado seguro). Ao detectar:

- Terminação de processo: matar um ou mais processos do ciclo (perda de trabalho)
- Preempção de recurso: forçar retirada de recurso de um processo (rollback necessário)

- Critério de escolha: menor custo — processo com menos trabalho feito, menos recursos retidos, menor prioridade

4. Ignorar (Ostrich Algorithm)

Simplesmente ignorar o problema — usado em sistemas onde deadlocks são raros e o custo de prevenir supera o custo ocasional de reiniciar. Abordagem do Linux e Windows para alguns recursos.

Comparativo das Estratégias

Estratégia	Quando age	Custo	Praticidade
Prevenção	Antes (projeto)	Alto — restrições severas	Baixa — muito restritivo
Esquiva	Em tempo real	Médio — $O(n^2)$ por req.	Média — requer max. prévio
Deteccção	Depois	Baixo preventivo, alto na recuperação	Alta — mais usado na prática
Ignorar	Nunca	Zero	Alta — sistemas não-críticos

SIMULADO

Questão 1 (Dissertativa)

Cite e explique as quatro condições necessárias para a ocorrência de deadlock (Condições de Coffman). Para cada condição, dê um exemplo concreto e indique como a negação dessa condição previne o deadlock.

Questão 2 (Dissertativa)

Explique o Algoritmo do Banqueiro de Dijkstra. O que é um 'estado seguro'? Por que o algoritmo é chamado de 'do Banqueiro'?

Questão 3 (Múltipla Escolha)

Qual estratégia de tratamento de deadlock é mais comumente usada em sistemas operacionais como Linux e Windows para recursos do sistema de arquivos? a) Prevenção — negar a condição de exclusão mútua b) Esquiva — Algoritmo do Banqueiro c) Detecção e recuperação periódica d) Ignorar (Ostrich Algorithm) — aceitar o risco de reinicialização

Questão 4 (Dissertativa)

Dois processos P1 e P2 precisam dos recursos R1 e R2. P1 adquire R1 e depois tenta adquirir R2; P2 adquire R2 e depois tenta adquirir R1. Explique como o deadlock surge aqui, identifique as 4 condições de Coffman presentes, e proponha uma solução.

Questão 5 (Dissertativa)

Compare as estratégias de prevenção e esquiva de deadlock. Em que cenários cada uma é mais apropriada? Quais as principais limitações de cada abordagem?

GABARITO COMENTADO

Questão 1 (Dissertativa):

1) Exclusão Mútua: recurso usado por 1 proc. por vez (mutex). Negar: tornar recurso compartilhável (nem sempre possível). 2) Hold & Wait: proc. segura recurso enquanto espera outro. Negar: requisitar tudo antes de iniciar. 3) No Preemption: rec. não pode ser retirado. Negar: preempção forçada. 4) Espera Circular: ciclo $P1 \rightarrow P2 \rightarrow P1$. Negar: ordenação global de recursos.

Questão 2 (Dissertativa):

Metáfora: banco (SO) empresta recursos (dinheiro) sabendo que clientes (processos) os devolverão. Estado seguro: existe sequência de processos onde cada um pode obter recursos necessários com os disponíveis + liberados pelos anteriores. Se atender requisição leva a estado inseguro, o SO bloqueia até ser seguro. Custo: $O(n^2 * m)$ por requisição.

Questão 3 (Múltipla Escolha):

d) Ignorar. Para a maioria dos recursos em SO de propósito geral, deadlocks são suficientemente raros que o overhead de prevenção/esquiva não se justifica. O usuário reinicia o processo se necessário.

Questão 4 (Dissertativa):

Deadlock: P1 segura R1 esperando R2; P2 segura R2 esperando R1 — espera circular. Condições: 1) R1 e R2 são exclusivos (mutex). 2) Cada proc. retém 1 rec. enquanto espera o outro. 3) Nenhum SO pode forçar liberação. 4) $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$ é o ciclo. Solução: ordenar recursos (sempre adquirir R1 antes de R2). Ambos tentariam R1 primeiro — um obteria, o outro esperaria sem deadlock.

Questão 5 (Dissertativa):

Prevenção: estática, impõe restrições rígidas (ex: ordenação de recursos). Simples de implementar, mas desperdiça recursos ou limita paralelismo. Adequada para sistemas embarcados com carga previsível. Esquiva: dinâmica, avalia cada requisição (Banker). Mais eficiente em uso de recursos, mas requer declaração antecipada da demanda máxima — impraticável em sistemas gerais onde a demanda não é conhecida a priori.